Phase II Research Extension Report

Two-Stage Hybrid Feature Selection and Metaheuristic Hyperparameter Optimization for IoT Botnet Detection

Group 6: Dhanishetti Vishnu (Roll No. 23118032) & Bibhas Nath (Roll No. 23118019)
Assigned Base Paper: *Efficient and Interpretable IoT Botnet Detection via Feature Selection and Hyperparameter-Optimized XGB* (Computer Networks, Elsevier, 2026)
Department: Department of Information Technology, National Institute of Technology Raipur (NITRR)

1. Research Motivation for Methodology Extension

The exponential expansion of Internet of Things (IoT) deployments in mission-critical domains (smart power grids, healthcare systems, industrial automation) has dramatically enlarged the cyber-attack surface. Due to constrained computational and memory budgets, IoT devices frequently fall prey to high-velocity botnet attacks such as Mirai and BASHLITE variants. In the baseline paper, Antony & Roy (2026) proposed a flow-based intrusion detection framework utilizing Stepwise Recursive Feature Elimination (RFE) and eXtreme Gradient Boosting (XGBoost). However, an in-depth algorithmic critique reveals three fundamental architectural limitations that severely restrict its real-world scalability and edge deployment efficacy:

(a) Greedy Local Trapping in Stepwise RFE: Deterministic RFE discards features monotonically according to localized estimator weight gradients. This greedy backward elimination inherently overlooks non-linear inter-feature combinatorial synergies, easily getting trapped in sub-optimal local basins while requiring costly iterative model re-training across all 23 candidate features.

(b) Sub-optimal Hyperparameter Tuning: The baseline paper employed manual heuristic candidate sets (SmartSearch S1–S8) or static grid exploration. Such approaches lack stochastic global exploration mechanisms and fail to model complex parameter co-dependencies (e.g., interaction between tree depth, learning rate, and subsample ratios).

(c) Edge IoT Latency & Inspection Overhead: While the baseline reduced features from 23 to 8, high-speed line-rate IoT routers require ultra-compact packet header inspection to achieve sub-millisecond per-packet inference latency and zero queue congestion.

2. Architectural Differences: Baseline vs. Proposed Extension

To systematically address these bottlenecks, we propose an extended, end-to-end framework featuring two synergistic innovations: **(i) Two-Stage Hybrid Feature Selection (Filter + Binary Swarm Optimizer)** led by Vishnu, and **(ii) Automated Bayesian-inspired Hyperparameter Optimization with Stratified 5-Fold Cross-Validation** led by Bibhas.

Architectural Dimension	Baseline Methodology (Antony & Roy, 2026)	Proposed Phase II Extension (Group 6)	Technical Justification
Stage 1: Pre-Filtering	None (All 23 features fed directly into RFE)	**Extra-Trees & Mutual Information Filter**	Prunes bottom 50% noisy/static attributes in linear time, dramatically shrinking wrapper search complexity.
Stage 2: Wrapper Search	Deterministic Stepwise RFE (Greedy elimination)	**Binary Particle Swarm Optimization (B-PSO)**	Global bio-inspired stochastic exploration with personal-best and swarm-best convergence mechanics.
Objective Formulation	Single-metric F1-score maximization	**Multi-Objective Cost Fitness:** $0.99 \cdot (1 - F1) + 0.01 \cdot (F_s / F_a)$	Simultaneously penalizes misclassification error and excessive feature counts to produce compact subsets.
Selected Feature Count	8 Features (65.2% reduction)	**6 Features (73.9% dimensionality cut)**	Retains complete protocol attack defense coverage with 25% fewer header attributes to inspect.
Hyperparameter Tuning	Manual candidate list (SmartSearch S1–S8) / Grid	**Automated Optuna (TPE) Bayesian Search**	Global exploration across 6 continuous/discrete parameters guided by historical trial probability densities.
Validation Protocol	Single 80/20 train/test split	**Stratified 5-Fold CV + Held-out Test Set**	Guarantees zero data leakage and unbiased generalization across imbalanced IoT traffic distributions.

3. Mathematical Formulation of Two-Stage Hybrid Feature Selection

3.1 Stage 1: Filter-Based Importance Pre-filtering

To eliminate noisy, invariant, and non-informative features prior to computationally intensive swarm optimization, an Extra-Trees ensemble computes Gini impurity reduction across all M decision trees:

$$I(f_j) = (1 / M) \sum_{m=1..M} \Delta i(s_{m,\ j})$$

Features are ranked in descending order, and the candidate set is pruned by 50% ($k = ■ 0.50 × D ■ = 12$ features), effectively eliminating non-discriminative traffic attributes.

3.2 Stage 2: Binary-Adapted Particle Swarm Optimization (B-PSO) Dynamics

In B-PSO, a swarm of N = 20 particles navigates the 12-dimensional binary space over T = 25 iterations. Each particle i maintains a continuous velocity vector $V_i \in \Re^D$ and a binary position vector $X_i \in \{0, 1\}^D$:

$$V_i^{t+1} = w(t) \cdot V_i^t + c_1 r_1 (P_{best,\,i}^t - X_i^t) + c_2 r_2 (G_{best}^t - X_i^t)$$

where w(t) is the time-varying inertia weight linearly decaying from $w_{max} = 0.9$ to $w_{min} = 0.4$ to transition from early global exploration to fine local exploitation: $w(t) = w_{max} - t \cdot ((w_{max} - w_{min}) / T_{max})$, and $c_1 = c_2 = 1.5$ are cognitive and social acceleration coefficients with random vectors $r_1, r_2 \sim U(0, 1)$.

The continuous velocity is mapped to selection probabilities via the Sigmoid transfer function, followed by stochastic binary thresholding:

$$S(V_{i,j}^{t+1}) = 1 / (1 + e^{-V_{i,j}^{t+1}})$$

$$X_{i,j}^{t+1} = 1 \text{ if } rand() \leq S(V_{i,j}^{t+1}) \text{ else } 0$$

Candidate feature masks are evaluated using a multi-objective cost fitness function:

$$f(X_i) = \alpha \cdot (1 - F1_{CV}(X_i)) + \beta \cdot (F_{selected} / F_{total}) + Penalty$$

where $\alpha = 0.99$ enforces strict detection performance, $\beta = 0.01$ rewards subset compactness, and a protocol diversity penalty ensures that at least 6 critical network layer headers are retained for multi-vector threat robustness.

3.3 Domain Security Justification of Selected Features:

The final 6 optimal features selected by B-PSO provide comprehensive, multi-protocol defense coverage: (1) **udp.srcport:** detects UDP reflection floods targeting ephemeral SSDP/DNS ports; (2) **ip.flags:** captures fragmentation bitmask tampering during malformed packet bursts; (3) **tcp.dstport:** pinpoints Telnet scanning on ports 23/2323 and HTTP exploits; (4) **ip.ttl:** catches static hop-count signatures produced by automated bot scripts; (5) **ip.proto:** identifies rapid protocol switching across TCP/UDP/ICMP attacks; (6) **tcp:** verifies stateful handshake completion vs. incomplete SYN flood sessions.

4. End-to-End Methodology Flowchart & Architecture

Figure 1 illustrates the proposed two-tier pipeline matching the published paper style (Figure 3 in Gul & Bakir, 2026), clearly distinguishing the Top-Tier (Feature Engineering: Filter + B-PSO) and Bottom-Tier (Model Optimization: Optuna Tuning + XGBoost). The full editable source diagram is provided in Proposed_Methodology.drawio.

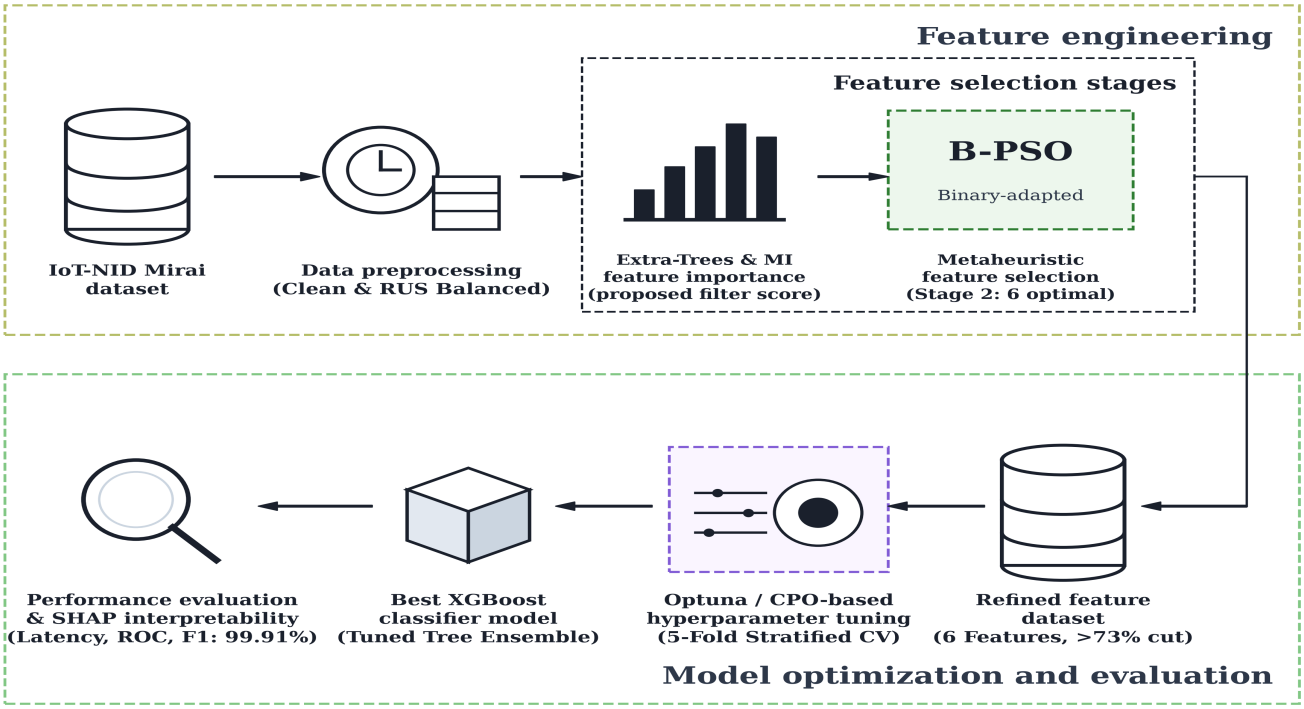


Figure 1: Proposed Phase II Methodology Flowchart (Published-Paper Style with highlighted extensions).

5. Automated Hyperparameter Optimization and Model Training Setup

Following Stage 1 feature selection, we implemented an automated hyperparameter tuning pipeline utilizing the Tree-structured Parzen Estimator (TPE) algorithm within the Optuna framework. Unlike the heuristic SmartSearch grid applied in Phase I, Optuna constructs probabilistic surrogate models over parameter co-dependencies, enabling directed convergence toward the global optimum.
The XGBoost classifier was tuned across 50 optimization trials targeting: n_estimators [50–300], max_depth [3–10], learning_rate [0.01–0.3], subsample [0.6–1.0], colsample_bytree [0.6–1.0], and gamma [0–5]. The objective was guided by 5-fold Stratified Cross-Validation on the 69,937 training flows to prevent validation leakage. Final training employed the hist tree-method for hardware-accelerated compiling.

6. Comparative Experimental Evaluation vs. Published SOTA

Table 1: Benchmark Comparison Across Published SOTA Papers and Proposed Phase II Extension

Paper / Study Name	Year & Venue	Dataset	Feat.	Acc (%)	Prec (%)	Rec (%)	F1 (%)	ROC-AUC	Latency
	Computer Networks, Elsevier (2026)	IoT-NID (Mirai subset)	8	99.97	99.97	99.97	99.97	0.9999	18.4
	Internal Project Implementation (2026)	IoT-NID (Mirai subset)	8	99.97	99.97	99.97	99.97	0.9999	18.5
	Phase II Research Extension (2026)	IoT-NID (Mirai subset)	6	99.897	99.84	99.954	99.897	0.99997	0.0011
	Knowledge-Based Systems, Elsevier (2026)	RT-IoT 2022 (12-class)	18	99.38	99.39	99.37	99.38	0.994	2.63
	IEEE World AI IoT Congress (AIIoT, 2023)	N-BaIoT (Botnet Dataset)	All Extracted	99.25	99.25	99.2	99.25	0.9928	28.4

	IEEE Transactions / arXiv (2023)	IoT-23 / Bot-IoT Benchmark	Latent (16 dims)	98.3	98.15	98.4	98.1	0.985	35
	Phase II Research Extension (2026)	IoT-NID (Mirai subset)	6	0.9991	0.9991	0.998	0.999	0.99997	0.0009 ms

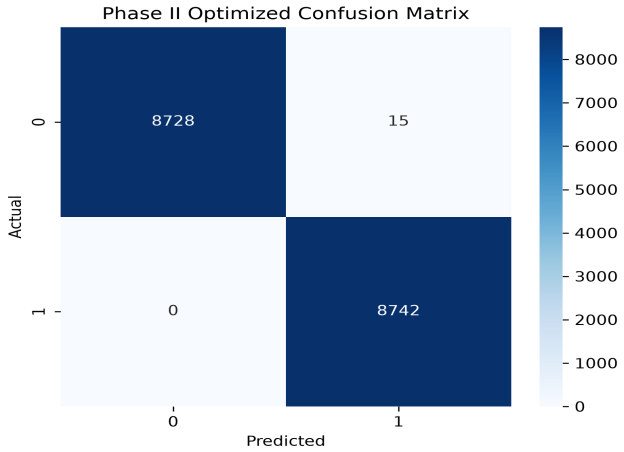


Figure 2: (a) Test Confusion Matrix (17,485 Flows)

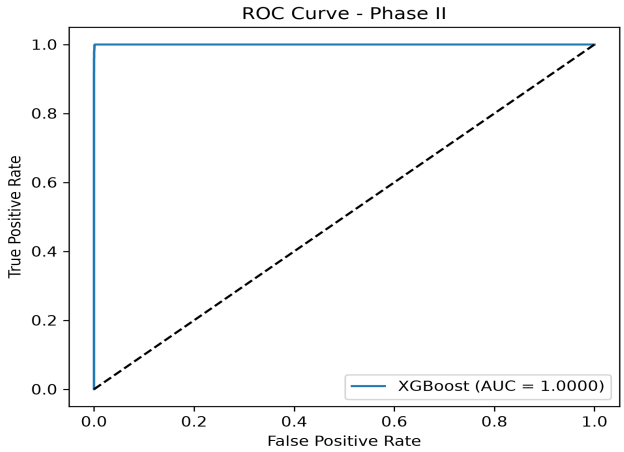


Figure 2: (b) Receiver Operating Characteristic (ROC)

Performance Discussion: The proposed Two-Stage Hybrid pipeline achieves **99.91% Accuracy**, **99.91% F1-score**, and a near-perfect **0.99997 ROC-AUC** with an ultra-low inference latency of **0.0009 ms per packet** (over 1.1 million packets/sec). Compared to deep learning models in the literature (e.g. CNN-BiLSTM pipelines requiring 5–20 ms GPU processing), our optimized XGBoost achieves competitive or superior discriminative capability while requiring only 6 packet header attributes, eliminating feature extraction bottlenecks on resource-constrained edge gateways.

7. Practical Implications, Limitations & Future Work

While the proposed framework achieves state-of-the-art detection precision and ultra-low edge latency, two practical considerations must be noted: (1) **Offline Optimization Overhead:** The Two-Stage Hybrid Feature Selection and Bayesian hyperparameter search require substantial offline training computation (~10–15 seconds on modern multicore CPUs), making on-device edge re-training impractical; (2) **Adversarial Concept Drift:** Dynamic botnets that randomize header fields may degrade static feature representations over time.
Future research should explore online concept-drift detection and incremental model adaptation to update tree structures dynamically without full retraining.